Problem A. Good Sequence

Time Limit 2000 ms

Problem Statement

You are given a sequence of positive integers of length N, $a = (a_1, a_2, ..., a_N)$. Your objective is to remove some of the elements in a so that a will be a **good sequence**.

Here, an sequence b is a **good sequence** when the following condition holds true:

- For each element x in b, the value x occurs exactly x times in b.

For example, $(3, 3, 3)$, $(4, 2, 4, 1, 4, 2, 4)$ and $()$ (an empty sequence) are good sequences, while $(3, 3, 3, 3)$ and $(2, 4, 1, 4, 2)$ are not.

Find the minimum number of elements that needs to be removed so that a will be a good sequence.

Constraints

- $1 \leq N \leq 10^5$
- a_i is an integer.
- $1 \leq a_i \leq 10^9$

Input

Input is given from Standard Input in the following format:

```
N
a_1 a_2 ... a_N
```

Output

Print the minimum number of elements that needs to be removed so that a will be a good sequence.

Sample 1

Input	Output
4 3 3 3 3	1

We can, for example, remove one occurrence of 3. Then, $(3, 3, 3)$ is a good sequence.

Sample 2

Input	Output
5 2 4 1 4 2	2

We can, for example, remove two occurrences of 4. Then, $(2, 1, 2)$ is a good sequence.

Sample 3

Input	Output
6 1 2 2 3 3 3	0

Sample 4

Input	Output
1 1000000000	1

Remove one occurrence of 10^9. Then, $()$ is a good sequence.

Sample 5

Input	Output
8 2 7 1 8 2 8 1 8	5

Problem B. Vector

Time Limit	1000 ms
Mem Limit	262144 kB
OS	Linux

For a dynamic array $A = \{a_0, a_1, ...\}$ of integers, perform a sequence of the following operations:

- pushBack(x): add element x at the end of A
- randomAccess(p):print element a_p
- popBack(): delete the last element of A

A is a 0-origin array and it is empty in the initial state.

Input

The input is given in the following format.

q
$query_1$
$query_2$
:
$query_q$

Each query $query_i$ is given by

`0` x

or

`1` p

or

`2`

where the first digits 0, 1 and 2 represent pushBack, randomAccess and popBack operations respectively.

randomAccess and popBack operations will not be given for an empty array.

Output

For each randomAccess, print a_p in a line.

Constraints

- $1 \leq q \leq 200,000$
- $0 \leq p <$ the size of A
- $-1,000,000,000 \leq x \leq 1,000,000,000$

Input	Output
8 0 1 0 2 0 3 2 0 4 1 0 1 1 1 2	1 2 4

Problem C. Jzzhu and Children

Time Limit	1000 ms
Mem Limit	262144 kB
Input File	stdin
Output File	stdout

There are n children in Jzzhu's school. Jzzhu is going to give some candies to them. Let's number all the children from 1 to n. The i-th child wants to get at least a_i candies.

Jzzhu asks children to line up. Initially, the i-th child stands at the i-th place of the line. Then Jzzhu start distribution of the candies. He follows the algorithm:

1. Give m candies to the first child of the line.
2. If this child still haven't got enough candies, then the child goes to the end of the line, else the child go home.
3. Repeat the first two steps while the line is not empty.

Consider all the children in the order they go home. Jzzhu wants to know, which child will be the last in this order?

Input

The first line contains two integers n, m ($1 \le n \le 100$; $1 \le m \le 100$). The second line contains n integers a_1, a_2, ..., a_n ($1 \le a_i \le 100$).

Output

Output a single integer, representing the number of the last child.

Examples

Input	Output
5 2 1 3 1 4 2	4

Input	Output
6 4 1 1 2 2 3 3	6

Note

Let's consider the first sample.

Firstly child 1 gets 2 candies and go home. Then child 2 gets 2 candies and go to the end of the line. Currently the line looks like [3, 4, 5, 2] (indices of the children in order of the line). Then child 3 gets 2 candies and go home, and then child 4 gets 2 candies and goes to the end of the line. Currently the line looks like [5, 2, 4]. Then child 5 gets 2 candies and goes home. Then child 2 gets two candies and goes home, and finally child 4 gets 2 candies and goes home.

Child 4 is the last one who goes home.

Problem D. Take ABC

Time Limit 2000 ms

Problem Statement

You are given a string S consisting of three different characters: `A`, `B`, and `C`.

As long as S contains the string `ABC` as a consecutive substring, repeat the following operation:

> Remove the leftmost occurrence of the substring `ABC` from S.

Print the final string S after performing the above procedure.

Constraints

- S is a string of length between 1 and 2×10^5, inclusive, consisting of the characters `A`, `B`, and `C`.

Input

The input is given from Standard Input in the following format:

```
S
```

Output

Print the answer.

Sample 1

Input	Output
BAABCBCCABCAC	BCAC

For the given string $S =$ `BAABCBCCABCAC`, the operations are performed as follows.

- In the first operation, the `ABC` from the 3-rd to the 5-th character in $S =$ `BAABCBCCABCAC` is removed, resulting in $S =$ `BABCCABCAC`.
- In the second operation, the `ABC` from the 2-nd to the 4-th character in $S =$ `BABCCABCAC` is removed, resulting in $S =$ `BCABCAC`.
- In the third operation, the `ABC` from the 3-rd to the 5-th character in $S =$ `BCABCAC` is removed, resulting in $S =$ `BCAC`.

Therefore, the final S is `BCAC`.

Sample 2

Input	Output
ABCABC	

In this example, the final S is an empty string.

Sample 3

Input	Output
AAABCABCABCAABCABCBBBAABCBCCCAAABCBCBCC	AAABBBCCC

Problem E. Train Ticket

Time Limit 2000 ms

Problem Statement

Sitting in a station waiting room, Joisino is gazing at her train ticket.

The ticket is numbered with four digits A, B, C and D in this order, each between 0 and 9 (inclusive).

In the formula $A\ op1\ B\ op2\ C\ op3\ D = 7$, replace each of the symbols $op1$, $op2$ and $op3$ with `+` or `-` so that the formula holds.

The given input guarantees that there is a solution. If there are multiple solutions, any of them will be accepted.

Constraints

- $0 \leq A, B, C, D \leq 9$
- All input values are integers.
- It is guaranteed that there is a solution.

Input

Input is given from Standard Input in the following format:

```
ABCD
```

Output

Print the formula you made, including the part `=7`.

Use the signs `+` and `-`.

Do not print a space between a digit and a sign.

Sample 1

Input	Output
1222	1+2+2+2=7

This is the only valid solution.

Sample 2

Input	Output
0290	0-2+9+0=7

$0 - 2 + 9 - 0 = 7$ is also a valid solution.

Sample 3

Input	Output
3242	3+2+4-2=7

Problem F. Balanced Brackets

OS Linux

A bracket is considered to be any one of the following characters: `(`, `)`, `{`, `}`, `[`, or `]`.

Two brackets are considered to be a *matched pair* if the an opening bracket (i.e., `(`, `[`, or `{`) occurs to the left of a closing bracket (i.e., `)`, `]`, or `}`) *of the exact same type*. There are three types of matched pairs of brackets: `[]`, `{}`, and `()`.

A matching pair of brackets is *not balanced* if the set of brackets it encloses are not matched. For example, `{[(])}` is not balanced because the contents in between `{` and `}` are not balanced. The pair of square brackets encloses a single, unbalanced opening bracket, `(`, and the pair of parentheses encloses a single, unbalanced closing square bracket, `]`.

By this logic, we say a sequence of brackets is *balanced* if the following conditions are met:

- It contains no unmatched brackets.
- The subset of brackets enclosed within the confines of a matched pair of brackets is also a matched pair of brackets.

Given n strings of brackets, determine whether each sequence of brackets is balanced. If a string is balanced, return `YES`. Otherwise, return `NO`.

Function Description

Complete the function *isBalanced* in the editor below.

isBalanced has the following parameter(s):

- *string s*: a string of brackets

Returns

- *string:* either `YES` or `NO`

Input Format

The first line contains a single integer n, the number of strings.
Each of the next n lines contains a single string s, a sequence of brackets.

Constraints

- $1 \leq n \leq 10^3$

- $1 \leq |s| \leq 10^3$, where $|s|$ is the length of the sequence.
- All chracters in the sequences $\in$ { **{**, **}**, **(**, **)**, **[**, **]** }.

Output Format

For each string, return `YES` or `NO`.

Input		Output

`STDIN`
`-----`
`3`
`{[()]}`
`{[(])}`
`{{[[(())]]}}`
	`Function`
	`--------`
	`n = 3`
	`first s = '{[()]}'`
	`second s = '{[(])}'`
	`third s ='{{[[(())]]}}'`
		`YES`
		`NO`
		`YES`

Explanation

1. The string `{[()]}` meets both criteria for being a balanced string.
2. The string `{[(])}` is not balanced because the brackets enclosed by the matched pair `{` and `}` are not balanced: `[(])`.
3. The string `{{[[(())]]}}` meets both criteria for being a balanced string.

Problem G. Together

Time Limit 2000 ms

Problem Statement

You are given an integer sequence of length N, $a_1, a_2, ..., a_N$.

For each $1 \leq i \leq N$, you have three choices: add 1 to a_i, subtract 1 from a_i or do nothing.

After these operations, you select an integer X and count the number of i such that $a_i = X$.

Maximize this count by making optimal choices.

Constraints

- $1 \leq N \leq 10^5$
- $0 \leq a_i < 10^5 (1 \leq i \leq N)$
- a_i is an integer.

Input

The input is given from Standard Input in the following format:

```
N
a_1 a_2 .. a_N
```

Output

Print the maximum possible number of i such that $a_i = X$.

Sample 1

Input	Output
7 3 1 4 1 5 9 2	4

For example, turn the sequence into $2, 2, 3, 2, 6, 9, 2$ and select $X = 2$ to obtain 4, the maximum possible count.

Sample 2

Input	Output
10 0 1 2 3 4 5 6 7 8 9	3

Sample 3

Input	Output
1 99999	1

Problem H. Permutations

Time Limit 1000 ms
Mem Limit 524288 kB

A permutation of integers $1, 2, \ldots, n$ is called *beautiful* if there are no adjacent elements whose difference is 1.

Given n, construct a beautiful permutation if such a permutation exists.

Input

The only input line contains an integer n.

Output

Print a beautiful permutation of integers $1, 2, \ldots, n$. If there are several solutions, you may print any of them. If there are no solutions, print "NO SOLUTION".

Constraints

- $1 \leq n \leq 10^6$

Example 1

Input	Output
5	4 2 5 3 1

Example 2

Input	Output
3	NO SOLUTION